

RIX2024 - Batch 1 - Finance Management System Roleplay Assessment

Team 2

W3H Analysis

Module 2: Registration & EMI Card Issuance	
What?	How?
1. What are the components involved in this module? A: The components involved in this module are: 1. Who am I? Component 2. User Application Component. 3. Admin Review Component. 4. User Onboard and Payment Component.	Who am I? Component: 1. Determine whether the user is a customer or a Customer Service Representative. Method 1: Role-Based API Response (Backend-driven) Method 2: Role Selection on UI (Frontend-driven) Method 3: Role Detection via Email Domain or Token Claims
2. What is the premises of this module? A: This module allows a user to onboard into any EMI Card plan as they desire once their initial documents are screened and approved by the admin.	User Application Component: 1. Allow user to input all their basic details. Method 1: Controlled React Form with State Management Method 2: Uncontrolled Form with Refs Method 3: Form Library Integration (e.g., Formik + Yup)
3. What are the functionalities of Who am I? Component? A: Who am I? Component: 1. Determine whether the user is a customer or a Customer Service Representative.	2. Allow users to upload their documents for id proofs. Method 1: File Upload via React Form to .NET API (Multipart/Form-Data) Method 2: Upload to Cloud Storage (e.g., Azure
4. What are the functionalities of User	

Application Component? A: User Application Component: 1. Allow user to input all their basic details. 2. Allow users to upload their documents for id proofs. 5. What are the functions of the Admin Review Component? A: Admin Review Component: 1. Allows admin to view all information submitted by user in a single dashboard. 2. Permits the admin to approve or reject any application of choice. 6. What are the functions of the User Onboard and Payment Component? A: User Onboard and Payment Component: 1. Allows user to choose their plan. 2. Allows user to pay the initial payment. 7. What are the different user roles interacting with this module? A: Customer, CSR, Admin 8. What data is collected during registration? A: Personal details, contact info, income details, document uploads. 9. What types of documents are required for verification? A: ID proof, address proof, income certificate 10. What validations are applied to the registration form? A: Mandatory fields, file type/size checks, format validations. 11. What happens after initial form submission? A: Admin review, eligibility check,	Blob or AWS S3) via Resigned URL Method 3: Base64 Encoding and Inline Submission Admin Review Component: 1. Allows admin to view all information submitted by user in a single dashboard. Method 1: Admin Dashboard with Aggregated API Data Method 2: Static Report Generation (PDF/CSV) Method 3: Database Viewer Tool Integration 2. Permits the admin to approve or reject any application of choice. Method 1: Admin Dashboard with Action Buttons (Approve/Reject) Method 2: Email-Based Approval Workflow Method 3: Manual Database Update via Admin Panel User Onboard and Payment Component: 1. Allows user to choose their plan. Method 1: Plan Selection UI with Backend API Integration Method 2: Plan Selection via Modal Popup with Local State Method 3: Plan Selection via Email Confirmation Link 2. Allows user to pay the initial payment. Method 1: Integrated Payment Gateway Method 2: Manual Bank Transfer with Upload Proof Method 3: QR Code-Based UPI Payment
---	---

approval/rejection. 12. What is the process for selecting and activating an EMI card? A: Card selection → Joining fee payment → Activation → Credit limit assignment. 13. What security measures are in place? A: Role-based access, encrypted uploads, secure payment gateway. 14. What are the edge cases or exceptions handled? A: Duplicate registrations, invalid documents, CSR misuse. 15. What is the expected user experience flow? A: From landing on registration page to card activation. 16. What is the fallback process if admin rejects the application? A: Notification, re-upload option, appeal process.	
Who am I? Component: 1. Determine whether the user is a customer or a Customer Service Representative. Method 1: Role-Based API Response (Backend-driven) 1. Secure and reliable: Role is determined and validated server-side, reducing spoofing risks. 2. Centralized logic: Keeps role management consistent across the system. 3. Scalable: Easily supports more roles or permissions in the future. User Application Component:	Who am I? Component: 1. Determine whether the user is a customer or a Customer Service Representative. Method 2: Role Selection on UI (Frontend-driven) 1. User-dependent: Relies on the user to correctly identify their role, which can be misused. 2. No verification: There's no backend validation to confirm if the user is truly a CSR. Method 3: Role Detection via Email Domain or Token Claims

1. Allow user to input all their basic details. Method 1: Controlled React Form with State Management - Simple and native: No external dependencies. - Full control: Easy to validate, manipulate, and track form data. - Ideal for small to medium forms, like basic user details. 2. Allow users to upload their documents for id proofs. Method 1: File Upload via React Form to .NET API (Multipart/Form-Data) - Simple and secure: Standard practice for file uploads in web apps. - Backend control: Allows validation, virus scanning, and secure storage. - Easier integration with existing .NET APIs and form handling. Admin Review Component: 1. Allows admin to view all information submitted by user in a single dashboard. Method 1: Admin Dashboard with Aggregated API Data - Real-time access: Admin sees the latest data instantly. - User-friendly: Custom UI tailored to admin needs. - Secure and scalable: Controlled access via role-based permissions. 2. Permits the admin to approve or reject any application of choice. Method 1: Admin Dashboard with Action Buttons (Approve/Reject) - Fast and intuitive: Admin can act directly from the dashboard. - Real-time updates: Immediate feedback and status change.	- Assumption-based: Email domain or token claims may not always accurately reflect the user's role. - Complexity: Adds unnecessary logic for parsing and validating domains or tokens. User Application Component: 1. Allow user to input all their basic details. Method 2: Uncontrolled Form with Refs - Less control: Harder to validate and manage dynamic form behavior. - Not scalable: Becomes messy with more fields or conditional logic. Method 3: Form Library Integration (e.g., Formik + Yup) - Overhead: Adds complexity and dependencies for a relatively simple form. - Better suited for large, complex forms with nested fields and advanced validation. 2. Allow users to upload their documents for id proofs. Method 2: Upload to Cloud Storage (e.g., Azure Blob or AWS S3) via Preassigned URL - Complex setup: Requires cloud configuration and permission management. - Overhead: Adds extra steps for generating and managing preassigned URLs. - Better suited for large-scale apps with heavy file storage needs. Method 3: Base64 Encoding and Inline Submission - Inefficient: Base64 increases file size (~33% overhead).
--	--

3. Secure and trackable: Actions are logged and validated via backend.

User Onboard and Payment Component:

1. Allows user to choose their plan.

Method 1: Plan Selection UI with Backend API Integration

1. Immediate and interactive: Users can select and confirm their plan in real-time.
2. Backend-driven: Ensures the selection is securely stored and validated.
3. Easily extendable: Supports dynamic pricing, discounts, or plan upgrades.

3. Allows user to pay the initial payment.

Method 1: Integrated Payment Gateway

1. Automated and secure: Real-time payment confirmation and fraud protection.
2. User-friendly: Seamless experience with minimal manual steps.
3. Scalable: Easily supports multiple payment methods and currencies.

2. Memory-heavy: Not ideal for large files or mobile devices.
3. Less secure: Harder to validate and manage files server-side.

Admin Review Component:

1. Allows admin to view all information submitted by user in a single dashboard.

Method 2: Static Report Generation (PDF/CSV)

1. Not interactive: Static reports don't support filtering or live updates.
2. Maintenance overhead: Requires regular generation and storage.

Method 3: Database Viewer Tool Integration

1. Technical barrier: Not suitable for non-technical admins.
2. Security risk: Direct DB access can expose sensitive data or allow unintended changes.

2. Permits the admin to approve or reject any application of choice.

Method 2: Email-Based Approval Workflow

1. Email dependency: Relies on email delivery and link security.
2. Not scalable: Difficult to manage large volumes of applications.

Method 3: Manual Database Update via Admin Panel

1. Risky: Direct database edits can lead to errors or inconsistencies.
2. Not user-friendly: Requires technical knowledge and lacks UI feedback.

User Onboard and Payment Component:

1. Allows user to choose their plan.

Method 2: Plan Selection via Modal Popup with Local State

1. Temporary state: Risk of losing selection if the session ends or page reloads.

	2. Delayed processing: Plan selection isn't confirmed until final submission. Method 3: Plan Selection via Email Confirmation Link 1. Disjointed UX: Takes users out of the app flow. 2. Email dependency: Relies on email delivery and user action outside the app. 2. Allows user to pay the initial payment. Method 2: Manual Bank Transfer with Upload Proof 1. Manual verification: Slower and error prone. 2. No real-time confirmation: Delays onboarding and card activation. Method 3: QR Code-Based UPI Payment 1. Limited automation: Requires manual entry or proof upload. 2. Not ideal for web-first apps: Better suited for mobile-heavy platforms.
Why?	Why Not?